

Audit interne complet & feuille de route d'optimisation

Projet « Lab Scheduling » — Universidad Loyola (ESI Sevilla)

Document d'audit technique — basé sur l'analyse du code source réel (`pipeline.py` , 4928 lignes) et des fichiers de données réels.

0. Résumé exécutif (TL;DR)

Question posée	Réponse courte
Peut-on faire « mieux que l'OPTIMAL » du CP-SAT ?	Oui — mais pas en touchant au solveur. Le statut <code>OPTIMAL</code> ne concerne QUE le placement des semaines sur un squelette figé . Le jour, le créneau, la salle et la composition des groupes sont décidés avant le solveur par une heuristique gloutonne. C'est là que se trouvent 80 % du potentiel d'amélioration.
Les jointures Excel sont-elles correctes ?	Globalement oui, avec une fuite silencieuse. La jointure <code>aulario</code> ≈ <code>alumnos</code> sur <code>MixtoID</code> couvre 332/337 groupes. 31 étudiants (≈3,2 %) sont perdus (969 → 938) faute d'horaire correspondant. Pas de log de réconciliation.
A-t-on la totalité des effectifs ?	Pour les labos : oui en pratique (les orphelins sont en aéro/cyber, hors labos). Mais le pipeline ne le prouve pas : aucun contrôle anti-fuite n'existe.
Causes d'INFAISABILITÉ ?	Quasi toujours structurelles : capacité salles × créneaux insuffisante, <code>MIN_GROUP_SIZE</code> , ordre strict des séances, et zones de blocage. Détaillées §4.
Les groupes OVERFLOW sont-ils bien gérés ?	Acceptable mais perfectible. Logique gloutonne « best-fit » avec recyclage. Risques : sous-optimalité, groupes sous le minimum laissés non affectés, pas de garantie de qualité. Alternatives §5.

1. Architecture réelle du système (ce que fait vraiment le code)

Contrairement à ce que le mot « solveur CP-SAT » laisse supposer, le système est un **pipeline en deux étages**, et le solveur n'intervient qu'au second :

```

ÉTAGE 1  HEURISTIQUE GLOUTONNE (form_groups, lignes 1036-1935)
  • Forme les groupes d'étudiants (taille 7..15)
  • Choisit le JOUR + le CRÉNEAU + la SALLE de chaque groupe
  • Applique la pénalité Vendredi, les préférences matin/aprèm
  • Gère les groupes OVERFLOW (exceptionnels)
  → Sortie : un squelette de groupes FIGÉS (jour/bloc/salle)

(jour, bloc, salle = CONSTANTES)

ÉTAGE 2  CP-SAT (solve, lignes 3088-3590)
  • UNE SEULE variable de décision : week_vars[session] = semaine
  • Décide en quelle SEMAINE tombe chaque séance
  • Objectif : étaler proprement les séances dans le calendrier
  → Statut OPTIMAL / FAISABLE / INFAISABLE

```

Implication majeure : quand le solveur renvoie `OPTIMAL`, cela signifie « étalement temporel optimal, étant donné un squelette jour/créneau/salle déjà fixé ». **Ce n'est PAS un optimum global du planning.** Le jour et le créneau — qui déterminent la charge du vendredi, l'utilisation des salles, les conflits — ne sont jamais optimisés mathématiquement.

1.1 Le modèle CP-SAT en détail (solve , l. 3129-3356)

Variable de décision unique

```

week_vars[s['id']] = model.NewIntVarFromDomain(
    cp_model.Domain.FromValues(valid_weeks), f"w_{s['id']}") # ligne 3141

```

Seule la **semaine** est une inconnue. `valid_weeks` exclut déjà les jours fériés (`HOLIDAYS`).

Contraintes DURES (hard)

Réf	Ligne	Signification	Type
C1	3147-3154	Deux séances même matière + même jour + même bloc → semaines différentes	!=
C4	3157-3169	Deux séances même salle + même jour + même bloc → semaines différentes (anti-collision salle)	!=
C5	3193-3201	Séances d'un même groupe strictement croissantes dans le temps ($S1 < S2 < S3...$)	>
C4-réservé	3172-3190	Créneaux salle réservés à une activité externe	souple (poids 100000)

Pénalités SOUPLES (objectif — `Minimize`, l. 3308-3335)

Terme	Poids	But
<code>first_excess</code>	100	Ne pas commencer trop tard dans la fenêtre
<code>last_deficit</code>	100	Ne pas finir trop tôt
<code>gap_deviations</code>	200	Espacement régulier entre séances
<code>parity</code> (alternance)	50	Alterner les groupes pairs/im-pairs
<code>c4_reserved</code>	100000	Éviter les créneaux salle réservés

Paramètres du solveur (l. 3342-3347)

```
solver.parameters.max_time_in_seconds = 300    # SOLVER_TIME_LIMIT
solver.parameters.num_search_workers    = 8
status = solver.Solve(model)
```

Constat — paramétrage perfectible :

- **✗** Pas de `relative_gap_limit` → le solveur peut « brûler » 300 s à prouver l'optimalité d'un écart

négligeable.

- ❌ Pas de `solver.parameters.random_seed` fixé → **résultats non reproductibles** d'un run à l'autre.
- ❌ Pas de warm-start / hints (`model.AddHint`) alors qu'on dispose d'une solution gloutonne — on jette une info gratuite.
- ❌ Pas de log de progression (`log_search_progress`) → difficile de diagnostiquer.

2. Peut-on optimiser « au-delà » de l'OPTIMAL CP-SAT ?

Réponse théorique : non, par définition. Un statut `OPTIMAL` du CP-SAT est une **preuve mathématique** qu'aucune meilleure valeur de l'objectif n'existe pour le modèle donné. On ne peut pas « battre » un optimum prouvé sans **changer le modèle**.

Réponse pratique : oui, largement — en changeant le modèle. Le potentiel d'amélioration ne vient pas de « mieux résoudre », mais de « mieux modéliser ». Trois leviers, par ordre d'impact :

Levier A — Internaliser le jour & le créneau dans le solveur ★★★★★ (impact maximal)

Aujourd'hui le jour/bloc sont figés par l'heuristique. En faisant du **jour et du créneau des variables de décision** (variables booléennes `x[groupe, jour, bloc]` avec contrainte « exactement un »), le CP-SAT optimise **globalement** :

- l'équilibrage du vendredi devient une **vraie contrainte souple** (et non un patch glouton),
- l'utilisation des salles est lissée par construction,
- on supprime des infaisabilités « artificielles » créées par de mauvais choix gloutons figés.

C'est la transformation d'un « **optimum local d'étalement** » en « **optimum global de planning** ». C'est le changement qui produit un vrai gain de qualité.

Levier B — Optimiser la composition des groupes ★★

La répartition étudiants→groupes est gloutonne (best-fit). Une formulation par affectation (ou set partitioning) réduirait le nombre de groupes overflow et lisserait les tailles (proche de `PRE-FERRED_GROUP_SIZE`).

Levier C — Affiner l'objectif & le solveur ★

- Ajouter `relative_gap_limit = 0.02` (s'arrêter à 2 % de l'optimum) → temps divisé, qualité quasi identique.
- `model.AddHint(...)` avec la solution gloutonne → convergence accélérée.
- `random_seed` fixé → reproductibilité (exigence d'audit).
- Objectif **lexicographique** ou pondérations normalisées : aujourd'hui les poids (100/200/50/100000) sont ad hoc ; un risque de « 1 unité de gap = 2 unités de retard » non voulu. Documenter et normaliser.

Conclusion §2 : Le bon message à l'utilisateur/commanditaire n'est pas « on a la solution optimale » mais « on a l'étalement optimal sur un squelette heuristique ». Le vrai gain qualité = **élargir le périmètre du solveur** (Levier A).

3. Audit interne des fichiers Excel (jointures & effectifs)

3.1 Les 4 fichiers et leurs rôles

Fichier	Rôle	Dimensions réelles	Clé
revi-sionAulario.xlsx	Horaires de TOUTE l'université (timetables)	13 423 lignes × 26 col, 2 640 activités	<code>mixtoID</code>
re-port_AlumnosGruposCentro-Decanos.xlsx	Inscriptions étudiants (enrollments)	8 216 lignes × 15 col, 337 activités, 969 étudiants	<code>MixtoID</code>
Asignacion_2025-2026_v5.xlsx	Profs & crédits (1 créd. P = 5 séances)	feuille « Asignacion docente » 307 × 89	<code>mixto ID / actividad ID</code>
informeDetalle-GruposPor-Curso.xls	Emplois du temps profs (créneaux libres)	format XML SpreadsheetML 2003	<code>Actividad + Docentes</code>

3.2 Audit de la jointure principale `aulario` ↔ `alumnos` (clé `MixtoID`)

Résultats mesurés sur les fichiers réels :

```

Aulario : 13 423 lignes – 2 640 MixtoID uniques (toute l'université)
Alumnos : 8 216 lignes – 337 MixtoID uniques – 969 étudiants uniques

MixtoID communs (jointure réussie) ..... 332 / 337 (98,5 %)
MixtoID inscription SANS horaire (orphelins) .. 5 → 27 étudiants
MixtoID horaire SANS inscrits (groupes vides) . 2 308 (= reste de l'université, NORMAL)

master_schedule.csv résultant : 32 424 lignes × 44 col – 938 étudiants uniques

```

✓ Ce qui est correct

1. **Clé de jointure pertinente** : `MixtoID` identifie sans ambiguïté le couple (activité × groupe).
2. **L'« explosion » est volontaire** : 8 216 inscriptions × N créneaux horaires (h1, h2...) → 32 424 lignes `slot_*`. C'est le dépivotage correct des horaires multiples.
3. **Les 2 308 groupes « sans inscrits »** ne sont PAS un bug : c'est le reste de l'université (autres facultés) que l'aulario contient mais que le pipeline filtre ensuite (`campus Sevilla` + mots-clés `LAB_CONFIG`).

⚠ Anomalies & risques détectés

1. **● Fuite silencieuse de 31 étudiants (969 → 938, ≈3,2 %)**. 27 sont des orphelins d'inscription (`MixtoID` sans horaire), dans des activités Aéronautique / Cybersécurité (« Introducción a la industria aeroespacial », « Tecnologías e infraestructuras de ciberseguridad »...). **Bonne nouvelle** : ce ne sont PAS des matières de labo → **les effectifs labos ne sont pas affectés en**

pratique. Mauvaise nouvelle : rien dans le code ne le vérifie ni ne le journalise. Une fuite future sur une vraie matière de labo passerait inaperçue.

2. ● **La colonne `aulario.alumnos` (effectif déclaré) est dénormalisée, non additive.** Sa somme (314 836) est dénuée de sens car le même effectif est répété sur chaque ligne d'horaire. Le code a raison de ne PAS l'utiliser : l'effectif réel = `nunique(AlumnoID)` du fichier inscriptions. À documenter pour éviter une régression.
3. ● **Aucune réconciliation explicite.** Le join réel (scripts amont « 01-02 ») **n'est pas dans le dépôt** — `master_schedule.csv` arrive pré-join. Donc l'audit ne peut pas vérifier le type de jointure (inner/left), ni si des doublons sont créés. **Recommandation : committer le script de jointure et y ajouter des assertions.**

3.3 Effectifs labos (sortie `lab_enrollments.csv`)

Les effectifs par matière de labo sont calculés par `identify_students` (l. 746-807) via mots-clés sur `actividad` + filtre `campus Sevilla` + exclusion Máster. Total mesuré : 22 matières, de 6 à 215 étudiants (Física II = 215, Física = 208, Química = 189...). **Cohérent.** Le calcul `dropna(['AlumnoID', 'slot...'])` (l. 831) écarte proprement les lignes sans créneau exploitable.

3.4 Côté professeurs : d'où viennent réellement les « créneaux libres » ?

Point d'attention : **le pipeline principal n'utilise PAS `informeDetalleGruposPorCurso.xls`** pour les disponibilités profs. `build_professor_busy` (l. 937-1035) dérive les créneaux occupés profs de la **même colonne `docentes` du `master_schedule`** (217 noms distincts). Le fichier `informeDetalle-GruposPorCurso` n'est lu que par l'utilitaire `build_subject_professors.py` (table matière→profs), pas dans le flux d'optimisation.

➡ **Conséquence :** la disponibilité d'un prof = « libre quand il n'a pas cours dans l'aulario ». C'est cohérent, mais cela suppose que **toutes** les charges du prof (réunions, autres campus, indisponibilités personnelles) figurent dans l'aulario — ce qui est **faux en général**. D'où l'intérêt de la nouvelle fonctionnalité « disponibilités enseignants » (PR #2) qui permet de saisir ces indisponibilités manuellement. **Bonne direction.**

4. Causes de FAISABILITÉ / INFAISABILITÉ

Le statut renvoyé (l. 3349) est `OPTIMAL`, `FAISABLE`, `INFAISABLE` ou `INCONNU`. Voici la **carte des causes**.

4.1 Pourquoi une instance est-elle FAISABLE ?

Une solution existe si, pour **chaque groupe**, on peut placer ses séances dans des semaines distinctes respectant :

- C5 (ordre strict $S1 < S2 < \dots$) → il faut **au moins autant de semaines disponibles que de séances** dans la fenêtre `[min_week, max_week]` (hors fériés).
- C1 + C4 → assez de « capacité semaine » sur le couple (salle/matière, jour, bloc) pour ne pas que deux groupes se télescopent.

4.2 Les 6 causes d'INFAISABILITÉ (par fréquence)

#	Cause	Où ça casse	Signature
1	Capacité salle×créneau saturée : trop de groupes sur (salle, jour, bloc) pour le nb de semaines	C4 (l.3164) + <code>can_fit_new_group</code> (l.1701)	INFEASIBLE ou groupes « skipped_full »
2	Fenêtre trop courte : <code>max_week - min_week + 1 < nb_séances</code> après retrait des fériés	C5 (l.3193) + <code>valid_weeks</code> vide (l.3137)	[WARN] aucune semaine disponible
3	MIN_GROUP_SIZE (7) non atteignable : moins de 7 étudiants mutuellement libres sur tout créneau	<code>form_groups</code> (l.1330, 1873)	étudiants laissés un-assigned
4	Conflit d'agendas étudiants : aucun créneau commun libre (tous occupés par d'autres cours)	<code>effective_busy</code> (l.1760)	groupe non formable
5	Zones bloquées / créneaux réservés trop nombreux	<code>is_week_blocked_for_session</code> (l.176), C4-réservé	pénalité explose ou domaine vide
6	Sur-contrainte des préférences profs (si converties en dures)	injection indispos (l.4514)	passse de FAISABLE à INFAISABLE

4.3 Le mécanisme de repli (model2, l. 3481-3562) — bonne pratique déjà présente

Le code possède un **second modèle de secours** : si le premier échoue ou est trop contraint, il **re-résout avec des contraintes relâchées**. C'est exactement la bonne approche (constraint relaxation). À renforcer (voir §6.4 : IIS / diagnostic d'infaisabilité).

4.4 Règle d'or

Une infaisabilité de ce système est presque toujours PHYSIQUE, pas algorithmique. 2 salles de Física × 5 jours × 6 blocs × N semaines = un plafond dur. Si l'effectif exige plus de groupes que ce plafond, **aucun solveur ne peut réussir** — il faut agir sur les ressources (salles, créneaux, fenêtre), pas sur le code. Le commentaire du code (l. 80-90) le reconnaît déjà pour le vendredi : les ~150 séances restantes sont « structurellement contraintes ».

5. Groupes en OVERFLOW (groupes exceptionnels) — audit & alternatives

5.1 Ce que fait le code aujourd'hui (l. 1779-1935)

1. **Recyclage d'abord** (l. 1736-1776) : avant de créer un groupe overflow, on tente de re-caser les étudiants non affectés dans les groupes existants non pleins (`refit`).  Bonne pratique.
2. **Création best-fit** : pour les restants (si $\geq \text{MIN_GROUP_SIZE}$), on crée des groupes supplémentaires en cherchant le créneau qui **maximise un score** :

```
python
score = len(free) - room_usage_penalty*5 - friday_penalty # l.1881
```
3. **Salles alternatives** (l. 1830) : Física/Química peuvent déborder vers « Ciencias Experimentales I/II ».
4. **Préférence matin/après-midi par année** (l. 1796-1805) avec fallback contrôlé (`AL-LOW_AFTERNOON_Y1Y3`).
5. **Garde-fou capacité** : `can_fit_new_group` (l.1701) vérifie C1 **et** C4 avant création → pas de sur-réservation salle. 
6. Les restants $< \text{MIN_GROUP_SIZE}$ sont **laissés non affectés** avec un warning (l. 1897).

5.2 Est-ce une « bonne pratique technique » ? — Verdict nuancé

Aspect	Évaluation
Recyclage avant création	 Correct (limite la prolifération de groupes)
Vérif capacité avant création	 Robuste (pas de collision physique)
Score glouton best-fit	 Sous-optimal & non garanti : un choix localement bon peut bloquer un meilleur global
Étudiants $< \text{MIN}$ laissés dehors	 Risque métier : des étudiants peuvent rester sans labo sans alternative proposée
Boucle <code>attempts < 50</code>	 Limite arbitraire ; peut s'arrêter avant d'avoir tout placé
Reproductibilité	 Dépend de l'ordre d'itération (<code>sorted_subjects</code>) → fragile

5.3 Alternatives recommandées (du plus simple au plus robuste)

Niveau 1 — Améliorer le glouton (effort faible)

- Trier les étudiants non affectés par **rareté de disponibilité** (les plus contraints d'abord) → réduit les « orphelins ».
- Remplacer la coupe sèche `< MIN_GROUP_SIZE` par une politique explicite : (a) fusion inter-programmes, (b) groupe dérogatoire validé manuellement, (c) liste d'attente. **Ne jamais perdre un étudiant en silence.**

Niveau 2 — Bin packing / heuristique de remplissage (effort moyen)

- Formuler l'overflow comme un **bin packing** (`salles × créneaux = bacs de capacité slot_capacity_for`) avec First-Fit Decreasing. Garantit un nombre de groupes proche de l'optimal théorique `[effectif / MAX_GROUP_SIZE]`.

Niveau 3 — Le solveur s'en charge (effort élevé, qualité maximale) ★

- Intégrer la **création de groupes dans le modèle CP-SAT** : nombre de groupes par matière = variable bornée par `min(n_profs, n_salles × capacité)`, affectation étudiant → groupe = variables booléennes, contrainte de capacité, et **minimisation du nombre de groupes overflow** dans l'objectif. → Plus de « best-fit glouton », un optimum prouvé qui n'abandonne jamais un étudiant tant qu'une place physique existe.

Niveau 4 — Robustesse réelle (production)

- **Matheuristique** : glouton pour une 1^{re} solution → injectée en hint CP-SAT → raffinement. Combine vitesse et qualité.
- **Analyse de scénarios** : « et si on ajoute 1 créneau le mardi après-midi ? » → quantifier le gain en étudiants placés (analyse de sensibilité, §6.5).

6. Bonnes pratiques & processus d'optimisation — méthode applicable

Voici le **processus standard** (recherche opérationnelle) pour un projet comme celui-ci, et comment l'appliquer ici.

6.1 Étape 1 — Formaliser le problème AVANT de coder

- **Ensembles** : étudiants, matières, groupes, salles, jours, blocs, semaines, profs.
- **Paramètres** : effectifs, capacités, fenêtres, fériés, indispos.
- **Variables de décision** : qu'est-ce qu'on choisit vraiment ? → ici, idéalement (groupe, jour, bloc, salle, semaine) — pas seulement la semaine.
- **Contraintes dures vs souples** : lister et **classer**. Document de référence versionné.
- **Objectif** : explicite, normalisé, hiérarchisé.

Application : le projet a sauté cette étape pour le jour/bloc (figés par habitude). La refaire débloque le Levier A (§2).

6.2 Étape 2 — Séparer données / modèle / solveur (déjà partiellement fait)

- **Couche données** (ETL) : jointures + **contrôles qualité obligatoires** (assertions anti-fuite : `assert n_in == n_out`).
- **Couche modèle** : construction du CP-SAT, testable isolément.

7. Plan d'action priorisé (quick wins → structurel)

Priorité	Action	Effort	Gain
 P0	Ajouter une assertion anti-fuite dans le join + journaliser les orphelins (les 31 étudiants)	Faible	Fiabilité données
 P0	Fixer <code>random_seed</code> + <code>relative_gap_limit=0.02</code> + <code>log_search_progress</code>	Très faible	Reproductibilité, vitesse
 P1	Politique explicite pour étudiants < MIN_GROUP_SIZE (jamais perdre en silence)	Faible	Risque métier
 P1	<code>model.AddHint()</code> avec la solution gloutonne (warm-start)	Faible	Convergence
 P1	Committer le script de jointure amont (01-02) + tests	Moyen	Auditabilité
 P2	Diagnostic d'infaisabilité (slack/IIS) systématique	Moyen	Maintenance
 P3	Levier A : faire du jour/bloc des variables CP-SAT	Élevé	Qualité globale ★
 P3	Overflow par CP-SAT (set partitioning)	Élevé	Zéro étudiant perdu

8. Conclusion

1. « **Mieux que l'optimal** » ? Pas en touchant au solveur — l' `OPTIMAL` est prouvé. Le vrai levier est d'**élargir le modèle** : aujourd'hui seul l'étalement des semaines est optimisé ; le jour, le créneau, la salle et les groupes sont décidés par une heuristique gloutonne **hors solveur**. Internaliser le jour/bloc (Levier A) transforme un optimum local en optimum global.

2. **Jointures Excel** : structurellement correctes (clé `MixtoID` , dépivotage volontaire), mais **fuite silencieuse de $\approx 3,2$ % d'étudiants** non contrôlée. Sans impact sur les labos cette fois (orphelins en aéro/cyber), mais à sécuriser par des assertions.
3. **Effectifs** : complets pour les labos en pratique ; à **prouver** par du code de réconciliation.
4. **Infaisabilité** : quasi toujours **physique** (salles $\times$ créneaux), pas algorithmique. Mécanisme de repli `model2` déjà présent — à systématiser avec un vrai diagnostic.
5. **Overflow** : logique gloutonne acceptable et prudente (vérif capacité), mais **sous-optimale et risquée** (étudiants laissés dehors). Migrer vers bin packing puis CP-SAT selon l'ambition.
6. **Processus** : appliquer la boucle Formaliser $\rightarrow$ QA données $\rightarrow$ Modéliser $\rightarrow$ Résoudre $\rightarrow$ Diagnostiquer $\rightarrow$ Mesurer $\rightarrow$ Sensibilité, avec reproductibilité et KPIs (le socle `reliability_metrics.py` existe déjà).

En résumé : le moteur est sain et la philosophie « contraintes dures + pénalités souples » est la bonne. Les gains se trouvent (a) en **élargissant le périmètre du solveur**, (b) en **blindant la couche données**, et (c) en **ne perdant jamais un étudiant en silence**.

Audit réalisé par analyse statique du code (`pipeline.py`) et analyse dynamique des fichiers de données réels (revisionAulario, report_AlumnosGruposCentroDecanos, master_schedule.csv, Asignacion_2025-2026_v5, informeDetalleGruposPorCurso).